Practical No. 9

C Program: Page Replacement Algorithms (FIFO and LRU)

1. AIM

To write a C program to implement the page replacement strategies FIFO and LRU, and to count the page faults produced by each.

2. REQUIREMENTS

- Operating System: Ubuntu / Linux (or Windows)
- Language: C, Compiler: GCC
- Text editor: Nano / Vim / VS Code
- Hardware: computer, keyboard, monitor

3. SHORT THEORY

Page replacement: when a page must be loaded into memory but all frames are full, the OS chooses an existing page to remove. The algorithm decides which one.

- Page: fixed-size block of a process's virtual memory.
- Frame: fixed-size slot in physical memory that holds a page.
- Page fault: the requested page is not in memory (it must be loaded). Fewer faults = better.
- Reference string: the sequence of pages requested by the process.
- FIFO (First In First Out): replaces the page that came into memory first (oldest). Works like a queue. Example: frames hold 1, 2, 3 and page 4 arrives, so page 1 is replaced. Weakness: may remove a page that is still used often (Belady's anomaly can occur).
- LRU (Least Recently Used): replaces the page not used for the longest time. Usually gives fewer page faults than FIFO but needs tracking of recent use.

4. ALGORITHM

FIFO:

- Enter reference string and number of frames.
- For each page, check if it is already in a frame. If yes, no fault.
- If not, put it in the oldest frame (circular pointer), and count a page fault.
- Display total faults.

LRU:

- Enter reference string and number of frames.
- For each page, check if it is in a frame. If yes, update its last-used time.
- If not, use an empty frame if there is one; otherwise replace the page with the smallest last-used time. Count a fault.
- Display total faults.

5. CODE (file name: page.c)

```
#include <stdio.h>

int main()
{
```

```
int n, f, i, j, choice;
int pages[50], frames[10];
int faults, pos, found;

printf("Enter number of pages: ");
scanf("%d", &n);

printf("Enter reference string:\n");
for(i = 0; i < n; i++)
    scanf("%d", &pages[i]);

printf("Enter number of frames: ");
scanf("%d", &f);

printf("1. FIFO\n2. LRU\n");
printf("Enter choice: ");
scanf("%d", &choice);

for(i = 0; i < f; i++)
    frames[i] = -1;

faults = 0;
pos = 0;

if(choice == 1)
{
    for(i = 0; i < n; i++)
    {
        found = 0;
        for(j = 0; j < f; j++)
            if(frames[j] == pages[i])
                found = 1;

        if(!found)
        {
            frames[pos] = pages[i];
            pos = (pos + 1) % f;
            faults++;
        }
    }
    printf("FIFO Page Faults = %d\n", faults);
}
else if(choice == 2)
{
    int last[50], time = 0, min, replace;

    for(i = 0; i < f; i++)
        last[i] = -1;

    for(i = 0; i < n; i++)
    {
        found = 0;
        time++;

        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                last[j] = time;
                break;
            }
        }

        if(!found)
        {
```

```
                replace = -1;
                for(j = 0; j < f; j++)
                    if(frames[j] == -1)
                    {
                        replace = j;
                        break;
                    }

                if(replace == -1)
                {
                    min = last[0];
                    replace = 0;
                    for(j = 1; j < f; j++)
                        if(last[j] < min)
                        {
                            min = last[j];
                            replace = j;
                        }
                }

                frames[replace] = pages[i];
                last[replace] = time;
                faults++;
            }
        }
        printf("LRU Page Faults = %d\n", faults);
    }
    else
    {
        printf("Invalid choice!\n");
    }
    return 0;
}
```

6. HOW TO EXECUTE (step by step)

Command / Key	What it does
`Ctrl + Alt + T`	Open the Ubuntu terminal.
`cd ~`	Go to the home folder.
`nano page.c`	Create the file, paste the code, save (Ctrl+O, Enter, Ctrl+X).
`gcc page.c -o page`	Compile the program.
`./page`	Run. Enter pages = 13, string = 7 0 1 2 0 3 0 4 2 3 0 3 2, frames = 3, choice = 1.
`./page`	Run again with the same input and choice = 2 (LRU).

7. LINE-BY-LINE EXPLANATION

Code	Meaning
`int n, f, i, j, choice;`	n = number of pages in the string, f = number of frames, i and j = loop counters, choice = FIFO or LRU.
`int pages[50], frames[10];`	pages[] = reference string. frames[] = the frames in memory.
`int faults, pos, found;`	faults = page fault count. pos = FIFO pointer (next frame to replace). found = 1 if page already in memory.
`scanf("%d", &pages[i]);`	Reads the reference string one number at a time.
`frames[i] = -1;`	-1 means the frame is empty at the start.

Code	Meaning
`faults = 0; pos = 0;`	Start with no faults and the pointer at the first frame.
`if(choice == 1)`	FIFO part starts here.
`found = 0;`	For each page, assume it is not in memory.
`if(frames[j] == pages[i]) found = 1;`	Search all frames. If the page is there, it is a hit.
`if(!found)`	Page not found, so a page fault occurs.
`frames[pos] = pages[i];`	Put the new page in the oldest frame.
`pos = (pos + 1) % f;`	Move the pointer to the next frame; % f wraps back to 0 after the last frame (circular queue).
`faults++;`	Count one page fault.
`else if(choice == 2)`	LRU part starts here.
`int last[50], time = 0, min, replace;`	last[] = last-used time of each frame. time = clock. min = smallest time. replace = frame to replace.
`time++;`	Clock moves forward with every page reference.
`last[j] = time; break;`	Page found (hit): update its last-used time and stop searching.
`if(frames[j] == -1) { replace = j; break; }`	If any frame is empty, use it first.
`if(replace == -1)`	No empty frame, so find the least recently used page.
`if(last[j] < min)`	Smaller time = used longer ago. The smallest one is the LRU page.
`frames[replace] = pages[i]; last[replace] = time;`	Put the new page there and set its last-used time.
`else { printf("Invalid choice!"); }`	If the user enters anything other than 1 or 2.

8. MEANING OF SYMBOLS

Symbol	Use
`!`	NOT. !found means found is 0 (page not found).
`% (modulus)`	Remainder after division. (pos+1) % f wraps the pointer around.
`-1`	Used as a marker meaning 'empty frame'.
`==`	Compares two values. (A single = assigns.)
`++`	Increase by 1.
`&`	Address-of, used in scanf.
`[ ]`	Array index.

9. SAMPLE INPUT AND OUTPUT

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 Frames = 3

Algorithm	Working (F = page fault)
FIFO trace	7[7] F, 0[7 0] F, 1[7 0 1] F, 2[2 0 1] F, 0 hit, 3[2 3 1] F, 0[2 3 0] F, 4[4 3 0] F, 2[4 2 0] F, 3[4 2 3] F, 0[0 2 3] F, 3 hit, 2 hit. Total = 10 faults
LRU trace	7[7] F, 0[7 0] F, 1[7 0 1] F, 2[2 0 1] F (7 oldest), 0 hit, 3[2 0 3] F (1 oldest), 0 hit, 4[4 0 3] F (2 oldest), 2[4 0 2] F (3 oldest), 3[4 3 2] F (0 oldest), 0[0 3 2] F (4 oldest), 3 hit, 2 hit. Total = 9 faults

Output: FIFO Page Faults = 10, LRU Page Faults = 9. (Verified by compiling and running the code.) LRU gives fewer faults here, so LRU performs better for this string.

10. CONCLUSION

Thus, the C program successfully implements FIFO and LRU page replacement. For the given reference string with three frames, FIFO produced 10 page faults and LRU produced 9 page faults, so LRU performed better.

11. QUICK VIVA ANSWERS

- Page replacement: replacing a page in memory when a new page must be loaded and no free frame is available.
- FIFO: replaces the page that entered memory first.
- LRU: replaces the page that has not been used for the longest time.
- Page fault: the requested page is not present in memory.
- Page frame: a fixed-size block of physical memory that stores a page.
- Which algorithm is used: both FIFO and LRU (chosen by the menu).
- Belady's anomaly: in FIFO, increasing frames can sometimes increase page faults. LRU does not have this problem.